Ghost Data Still Exists

A plain-language guide to the deletion-audit agent built for HackerRank Orchestrate (October 2026)

In one sentence: when a customer asks a company to delete their data, this program checks every place that data could hide, decides whether it is *really* gone, and, if not, writes the exact list of things to fix and how to prove they were fixed.

Section	What it covers
1. The problem	The story behind the challenge and why "Complete" can be a lie
2. Key words	A short glossary so the rest reads easily
3. Inputs and outputs	What the program reads and what it must produce
4. How the agent works	The 8 stages, one at a time, in simple terms
5. The rules	What each kind of data must look like after deletion
6. Worked examples	Two real sample cases, step by step
7. Where AI is used	Only two jobs, and why; cost and token use
8. Results	How accurate it is, how it was tested, what it still misses
9. Running it	Setup, commands, files, caches
10. Submission	What gets handed in and where

Northstar Shop and every person, email and record in this project are fictional.

1. The problem

Northstar Shop is an imaginary online store. A customer asked to have their account deleted, and every internal dashboard said the deletion was **"Complete"**. Weeks later the same person tried to sign up again and got: "*This email is already registered.*"

Digging in, the company found the person's data was still sitting in many places: copies of the database, support ticket attachments, the search index, analytics exports, log files, archives and backups. Worse, an automatic sync job had **re-created** data that had already been deleted. The failure cost $2.4 million.

The lesson: a deletion system saying "done" is just a **claim**. You need an auditor that looks at the actual evidence. That auditor is what this project builds.

What the agent has to do for each request

1. **Decide if it is really a deletion request.** People write in many languages and ask for many things (a copy of their data, a name correction, no more newsletters). Only real deletion requests get audited.
2. **Work out who the person is**, including old email addresses and phone numbers they used to have, because old systems may still store the old ones.
3. **Find every record about them** across all systems and all copies.
4. **Work out the true state of each record** at a specific moment (the case's `audit_at` time), ignoring anything that happened later.
5. **Apply the company's deletion policy.** Not everything should be deleted: some records must be kept, some just have names and emails blanked out, some are made anonymous.
6. **Give a verdict and a plan:** COMPLETE (prove it), INCOMPLETE (list problems, the fix plan, and the checks), or NOT_APPLICABLE (not a deletion request).

A deletion is COMPLETE only when all four are true: every copy got the treatment the policy requires; no sync, import or backup restore can bring the data back; records that are kept show only the identifiers the policy allows; and records the policy says to keep are still there.

2. Key words

Word	Plain meaning
Case	One deletion request. There are 518 to answer, plus 32 practice cases with known answers.
audit_at	The moment the answer must describe. Evidence from after this time must be ignored.
reported_status	What the company's own deletion system *claims* (COMPLETE, PARTIALLY_COMPLETE, NOT_STARTED). Never trusted as proof.
Identifier	Anything that points to a person: name, email, phone, an email hash, or the internal `user_id`.
Email hash	A scrambled fingerprint of an email address. Still counts as personal data because it can be matched back.
Resource	One record in one system, e.g. `account_7566` in the account service.

Word	Plain meaning
Copy / replica	The same database kept in more than one place (main, read replica, failover replica). A record deleted from the main copy can survive in a replica.
Data flow	An automatic job (sync, import, export) that can copy data or re-create it.
Restore risk	A backup that could be restored and bring deleted data back.
Tombstone	What is left of a deleted account so other kept records still make sense: only the internal `user_id`, with name, email, phone and hash removed.
Redact	Keep the record but blank out the personal fields.
Anonymize	Keep the record but cut every link to the person, including the `user_id`.
Retain	Must be kept as-is (e.g. security logs within their retention period, records under legal hold).
Evidence id	The id of the file, activity row or audit row that proves a finding, e.g. `evidence_94`.
Sighting	This project's word for one observation of one record in one piece of evidence.

3. Inputs and outputs

What the program reads (all inside `dataset/`)

File	What it holds
deletion_requests.csv	The 518 requests to answer: email, dates, the system's claimed status, the audit time, and the request text.
sample_deletion_requests.csv + sample_output.csv	32 practice requests with the correct answers. Used only to measure accuracy.
user_identities.csv	Each person's current details: user_id, name, email, phone, email hash.
user_audit_log.csv	History of changes to those details, so old emails and phones can be found.
system_activity.csv	About 4,000 events from all systems: deletions, syncs, exports, backups, restores, indexing. Mixes many people, and its "result" column is only a claim.
deletion_rules.csv	The company policy: 15 rules saying what to do with each kind of data.
evidence_files.csv + evidence/	94 evidence files: SQL database snapshots, JSON, logs, text records, PDFs and screenshots. The list does not say which person a file is about; the program must work that out from the contents.

What the program writes

One file, `output.csv`, at the repository root, with exactly one row per case, sorted by case id:

```
case_id,deletion_status,steps_json
case_000,NOT_APPLICABLE,[]
case_001,COMPLETE,[{...},{...}]
```

`steps_json` is an ordered list of steps. Every step says which **system** and **record** it is about, what **operation** to do, the resulting **state**, and the **evidence** that proves it. Steps come in three phases, always in this order:

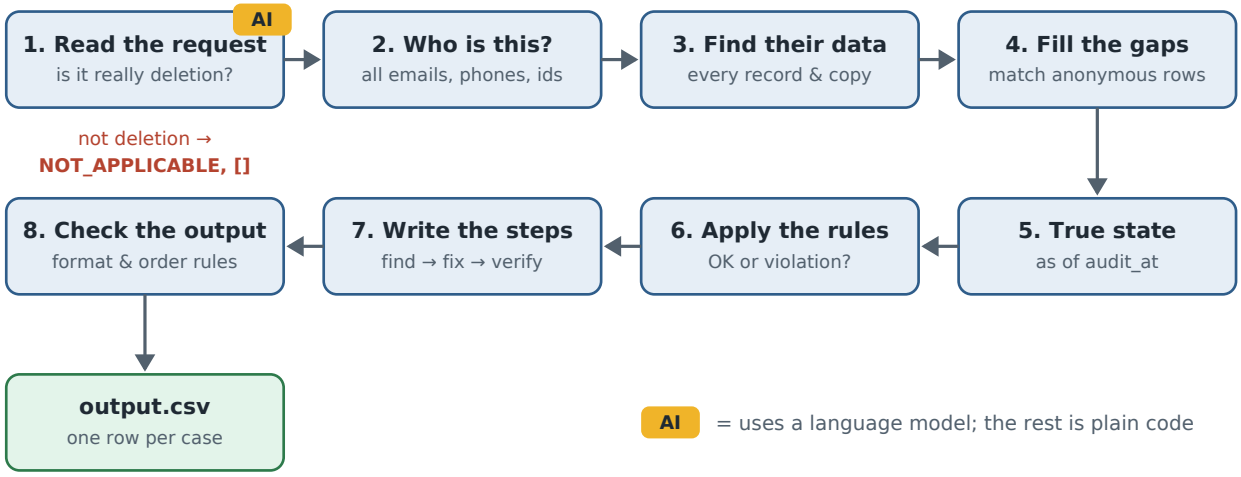
Phase	Question it answers	Operations
FINDING	What is wrong right now?	REPORT_REMAINING_RECORD, REPORT_EXPOSED_PII, REPORT_ACTIVE_DATA_FLOW, REPORT_RESTORE_RISK
REMEDIATION	What must be done to fix it?	DISABLE_DATA_FLOW, BLOCK_RESTORE, DELETE_RECORD, REDACT_RECORD, ANONYMIZE_RECORD
VERIFICATION	How do we prove it is right?	VERIFY_RECORD_ABSENT / PRESENT / REDACTED / ANONYMIZED, VERIFY_DATA_FLOW_DISABLED, VERIFY_RESTORE_BLOCKED

Verdict	What the step list contains
COMPLETE	Only verification steps, each citing the evidence that proves the record is in the right state.
INCOMPLETE	Findings (citing evidence), then the full fix plan, then the checks to run afterwards. Fix and check steps have an empty evidence id because they describe future work.
NOT_APPLICABLE	An empty list `[]`.

Ordering matters. Inside each phase, steps follow the operation order in the table above, then system id, then record id. A handy side effect: stopping sync jobs and blocking backups always comes *before* deleting records, so nothing gets re-created halfway through the fix.

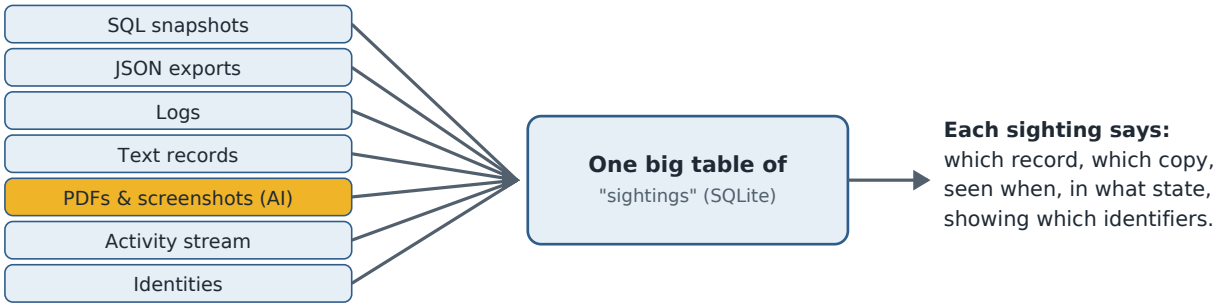
4. How the agent works

The program is written in Python. It first loads every file into a small local database, then runs each case through the same 8-stage workflow (built with LangGraph). The big picture:



Before any case: loading the evidence

Evidence comes in many formats. A separate loader for each format turns it into the same shape: a **sighting**. All sightings go into one table, so later stages never need to care whether a fact came from a SQL dump or a screenshot.



Format	How it is read
SQL snapshots	Executed in a throwaway SQLite database and inspected. Main and replica tables are kept as separate copies. A table's full listing also proves *absence*: if the record is not in it, it was not there.
JSON, text, logs	Parsed with ordinary code. Log lines are joined by trace id, so a "not found" line knows which identifier was searched.
PDFs and screenshots	Read once by a language model that returns structured fields. For PDFs, every identifier the model reports is checked against the PDF's real text, so a made-up value can never link evidence to the wrong person.
Activity stream	Results like "deleted successfully" are stored as **claims** with no state. Only status checks ("remains enabled", "can still be restored") count as state.
Identities	Every current and past email, phone and hash for each user, with the time window each was valid. Hashes are computed for old emails too.

Stage 1 — Read the request (uses AI)

- A language model reads each request in whatever language it was written and labels it: DELETION, ACCESS, PORTABILITY_EXPORT, CORRECTION, MARKETING_OPT_OUT or OTHER.
- "Please check my earlier deletion and finish it" counts as deletion. "Unsubscribe me, but keep my account" does not.
- Non-deletion requests stop here with **NOT_APPLICABLE** and `[]`. In the final run, 10 of 518 requests were non-deletion.
- As a safety check, the answers are compared with the company's own intake log. They agreed on all 10. Disagreements would only be printed as warnings; the log never overrides the model.

Stage 2 — Who is this?

- The request email may be current or old. It is looked up in both the identity snapshot and the change history to find the stable `user_id`.
- From that, the program collects every email, phone number and hash the person ever had.
- If the same person made an earlier request, evidence up to that earlier request's audit time belongs to the earlier episode and is left out, so an old, already-handled account is not reported again.

Stage 3 — Find their data

- Every sighting that shows one of the person's identifiers, and was seen by the audit time, links that record to the case.
- This finds records directly, including those in replicas, logs, documents and exports.

Stage 4 — Fill the gaps (cross-case matching)

- The hard part: once a profile is deleted, or an analytics record is anonymized, it no longer carries any identifier. The only trace is an anonymous row from the deletion worker in the activity stream.
- These rows are matched to cases **by elimination across all 550 requests at once**, like a logic puzzle. Clues used: each kind of row appears in a typical time window after the case's report time; the row's result must agree with the case's claimed status; each case owns at most one row of each type; rows from one workflow run follow a fixed seconds-past-the-minute rhythm (learned from cases already solved for certain, never hard-coded).
- A row is assigned only when exactly one case is left. If two cases could still own it, it stays unassigned. Guessing would risk inventing wrong records.

Stage 5 — State at audit time

- For each record and each copy, the program picks the latest trustworthy sighting at the audit time: PRESENT, ABSENT, REDACTED, ANONYMIZED, ACTIVE, DISABLED or BLOCKED.
- Claims never prove a state; only evidence does. Each copy is judged on its own, so a stale replica row is still a problem even when the main copy is clean.
- A record missing from a full listing is only absent *by inference*. A direct sighting during the request's lifetime beats a later listing that leaves the record out.
- For anything still present, it records exactly which of the person's identifiers are still visible.

Stage 6 — Apply the rules

- Each record gets a data category (from the evidence's own record type, or from its id prefix and fields). The category picks the rule; the rule decides if the record is COMPLIANT, a VIOLATION, or UNKNOWN.
- Any violation makes the whole case INCOMPLETE. See section 5 for the rules.

Stage 7 — Write the steps

- Violation: a finding (citing the evidence), a fix, and a check.
- Compliant record in a COMPLETE case: one check citing the proof.
- Compliant record in an INCOMPLETE case: one check with no evidence, because it runs after the fix.
- Steps are sorted by phase, operation, system and record. JSON is written with sorted keys so reruns produce identical bytes.

Stage 8 — Check the output

- A validator checks every case against the contract: steps numbered 1, 2, 3...; phases and operations in order; only allowed operation/state pairs; evidence present where required and empty where required. The final run had 0 problems.

5. The rules: what "deleted" means for each kind of data

The policy file has 15 rules. They boil down to the treatments below. The last column says when a record counts as done.

Treatment	Applies to	Counts as done when...
DELETE	User profile, secondary identifiers, search index entries, cache entries, user exports, marketing profiles, support attachments that belong only to this person	the record is gone from **every** copy
REDACT	Support messages with shared context (other people, orders), email delivery logs	the record is kept but has no name, email, phone or hash (the customer `user_id` may stay)
REDACT to tombstone	The deleted account itself	only the stable `user_id` remains
ANONYMIZE	Analytics profiles, product telemetry	the record is kept with nothing linking it to the person, not even `user_id`
RETAIN	Security audit logs within retention, records under an active legal hold	the record is still there
RETAIN + block restore	Backup archives	the backup is kept but cannot restore this person's data
Disable	Sync, import and export jobs that can re-create data	the job is disabled

Why not just delete everything? Because the policy forbids it. Deleting a security log that must be kept for a year, or a record under legal hold, is a violation too. The agent has to get the treatment right, not only find the data.

6. Worked examples

Example A — case_03, a deletion that really is COMPLETE

The request (in French) asks for permanent deletion. One evidence file captured before the audit time (`evidence_94`) shows the account is gone, the security audit record is still kept as required, and the analytics record is anonymized. So the answer contains only proof:

#	Phase	System / record	Operation	Evidence
1	VERIFICATION	account_service / account_7566	VERIFY_RECORD_ABSENT	evidence_94
2	VERIFICATION	security_service / audit_record_2016	VERIFY_RECORD_PRESENT	evidence_94
3	VERIFICATION	analytics_platform / analytics_9406	VERIFY_RECORD_ANONYMIZED	evidence_94

Example B — case_09, the system said "not started", and something is still wrong

The account is already gone and the security record is kept correctly. But evidence `evidence_93` shows a backup that can still be restored, which would bring the account back. So the case is **INCOMPLETE**:

#	Phase	System / record	Operation	Evidence
1	FINDING	backup_service / backup_4978	REPORT_RESTORE_RISK	evidence_93
2	REMEDIATION	backup_service / backup_4978	BLOCK_RESTORE	(future)
3	VERIFICATION	account_service / account_3960	VERIFY_RECORD_ABSENT	(future)
4	VERIFICATION	security_service / audit_record_4902	VERIFY_RECORD_PRESENT	(future)
5	VERIFICATION	backup_service / backup_4978	VERIFY_RESTORE_BLOCKED	(future)

Notice that the checks cover the *whole* required end state (account absent, audit record present, backup blocked), not only the record that was fixed. "(future)" means the evidence id is an empty string.

7. Where AI is used, and why so little

A language model is used for exactly two jobs, the ones ordinary code cannot do well. Everything else is plain code, so it is fast, free to rerun, and gives the same answer every time.

Job	Why AI	How it is kept safe and cheap
Classify the request	Requests are free text in many languages, with subtle wording ("verify my deletion" vs "keep my account").	Identical texts are sent once; 20 texts per call; results cached on disk. Answers must fit a fixed schema.
Read PDFs and screenshots	Invoices, support records and app screens need reading, not parsing.	One call per file, cached. PDF identifiers are checked against the real PDF text before use.

Model and cost of the final full run

Item	Value
Provider / model	Anthropic, claude-sonnet-5-5 (configurable through the MODEL setting; works with any LangChain provider)
Model calls	35 (20 classification + 15 document extraction)

Item	Value
Tokens	123,413 input + 29,828 output = 153,241 total
Average per request	about 296 tokens
Estimated cost	about USD 0.55 in total, about USD 0.001 per request
Time	about 3 minutes cold; a few seconds when caches exist (and then 0 model calls)

Caches change nothing about the answer. A run with every cache deleted and a run using the caches produced byte-identical `output.csv` files. Cache keys include the text, the prompt and the model, so editing a prompt or switching model automatically refreshes the results.

8. Results

Accuracy on the 32 practice cases

Metric	Score	Meaning
Status accuracy	100% (32/32)	COMPLETE vs INCOMPLETE right on every case
Step precision	100%	every step the agent wrote was correct
Step recall	97.4%	it found 188 of 193 expected steps
Step F1	98.7%	precision and recall combined
Field accuracy	97.4%	system, state and evidence correct (step number 92.2%, because a missing step shifts later numbers)
Exact full match	84.4% (27/32)	the whole step list identical, in order
Sequence similarity	96.3%	how close the order is, on average

Extra tests written by hand

- **8 audit cases**, each a tiny made-up world run through the real workflow: stale vs fresh sightings, replica drift, evidence after the audit time, tombstone + retention + blocked backup, recreation risks ordered first, a repeat request by the same person, and a non-deletion request. All 8 pass.
- **6 classifier cases** in English, German and Spanish (unsubscribe, delete, verify-and-finish, name correction, close but keep invoices, data copy first). All 6 pass.
- To prove the tests actually catch mistakes, two behaviours were switched off on purpose; the matching tests failed as they should.

Final answers on the 518 real cases

COMPLETE: 155 · INCOMPLETE: 353 · NOT_APPLICABLE: 10 · contract problems: 0.

How it got there

Seven rounds of improvement, each measured on the practice cases. Recall went from 76.7% to 97.4%, and exact matches from 28% to 84%. The biggest gains came from: trusting fresh direct sightings over later partial listings; dating each fact by when it happened rather than when the file was exported; and matching anonymous deletion rows by elimination, then sharpening that with the learned timing rhythm. One change was rolled back after it changed answers on the real cases for the wrong reasons.

What it still misses

- 3 expected records appear nowhere in the provided data except the answer key, so no program could find them.
- 2 anonymous deletion rows could still belong to more than one case after all the clues. They are left out on purpose rather than guessed.
- Some rules (listing precedence, category hints) were learned from a handful of practice cases; on new data they are the most likely source of error.

9. Running it

Setup (once)

1. Install **uv** (a Python package manager). It also downloads Python 3.12 if needed.
2. From the repository root, install dependencies: `uv sync --project code`
3. Create a `.env` file in the repository root with `MODEL=anthropic:claude-sonnet-5-5` and your API key. The file is git-ignored and must never be committed or zipped. No key is needed if the caches in `code/.cache/` already exist.

Commands (run from the repository root)

Command	What it does
uv run --project code python code/main.py run	Answers all 518 requests and writes `output.csv` plus the usage report
... main.py sample	Answers the 32 practice requests and scores them
... main.py evaluate	Practice run and score, plus the 8 + 6 hand-built tests
... main.py score	Re-scores an existing practice output without rerunning
... main.py ingest	Only rebuilds the evidence database (for debugging)
add --no-llm	Forbids model calls; uses caches only and stops clearly if something is missing

Every run rebuilds the database from `dataset/`, so old data can never leak into an answer. Each run also writes a trace per case to `code/.cache/traces/` showing exactly why each decision was made: the intent, the identity found, every linked record with its rule, verdict and evidence.

Where things live

Path	Contents
code/main.py	The command-line entry point
code/audit/ingest/	One loader per evidence format
code/audit/intent.py, extraction.py, llm.py	The two AI jobs and the provider-neutral model wrapper
code/audit/identity.py, discovery.py	Stages 2 and 3
code/audit/correlation.py	Stage 4, cross-case matching
code/audit/reconstruct.py, policy.py	Stages 5 and 6
code/audit/steps.py	Stages 7 and 8: build, order, validate and serialize steps

Path	Contents
code/audit/graph.py, pipeline.py	The LangGraph workflow and the end-to-end run
code/audit/scoring.py, usage.py	Accuracy scoring and the token/cost report
code/prompts/	The two prompts sent to the model
code/evaluation/	Hand-built tests, evaluation report, usage report
code/.cache/	Database, AI caches, usage log, traces (git-ignored, not zipped)

10. Submission

Item	What it is
code.zip	The code, prompts, configuration, README and the evaluation folder. No dataset, virtual environment, caches or secrets.
output.csv	The 518 answers.
Chat transcript	The root `log.txt` written by the coding agent during development.
AI interview	Explaining the architecture, investigation, remediation planning and evaluation; this document is a good starting point.

Submit at: https://www.hackerrank.com/contests/hackerrank-orchestrate-october26/challenges/ghost-data-still-exists

The whole idea in five lines. Don't trust "Complete"; look at evidence. Find the person under every name they ever used. Check every copy, not just the main one. Delete, redact, anonymize or keep exactly as the policy says. Stop anything that could bring the data back before cleaning up, then prove the result.